LAPORAN PRAKTIKUM
PEMROGRAMAN BERORIENTASI OBJEK
MODUL 4 – Enkapsulasi



NAMA: ANDITA AYUNINGTYAS

NIM: 103132400016

KELAS: PS1DS-05-REG-01

PROGRAM STUDI S1 SAINS DATA

FAKULTAS INFORMATIKA

TELKOM UNIVERSITY PURWOKERTO

2026

A. Dasar Teori

Dalam bahasa pemrograman Java, enkapsulasi sangat erat kaitannya dengan penggunaan *modifier* untuk memberikan batasan akses pada kelas, *interface*, *method*, maupun variabel. Untuk menerapkan penyembunyian data (*data hiding*) dengan baik, atribut dalam suatu kelas umumnya diberikan *modifier private*, sedangkan *method* perantaranya menggunakan hak akses *public*. Selain *access modifier* biasa, Java juga memiliki tipe *modifier* lain untuk keperluan spesifik, seperti *abstract* dan *transient*.

Untuk menginstansiasi atau menciptakan sebuah objek baru, bahasa pemrograman ini menggunakan struktur khusus bernama *constructor*. *Constructor* merupakan sebuah *method* khusus yang namanya wajib sama persis dengan nama kelasnya dan sama sekali tidak mengembalikan nilai (*return value*). Selanjutnya, kelas-kelas yang telah dibangun dapat diatur dan dikelompokkan ke dalam sebuah *package*. Penggunaan *package* ini sangat penting terutama jika kelas yang dikembangkan sudah cukup banyak, sehingga dapat mempermudah tata kelola program sekaligus mencegah terjadinya konflik penamaan kelas.

B. Guided

1. [Manusia]

```
 /*
 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this license
 * Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Class.java to edit this template
 */
package andita.modul04.guided1.manusia;


public class Manusia {
//definisi atribut


    private String nama;
    private int umur;


    // 4.2 constructor
```

```
    public Manusia(){};
    public Manusia(String nama){
        this.nama =nama;
    }

    public Manusia(String nama, int umur){
        this.nama = nama;
        this.umur = umur;
    }

//definisi Method
//method setter
    public void setNama(String a) {
        nama = a;
    }

//method getter
    public String getNama() {
        return nama;
    }

 //method setter
    public void setUmur(int a) {
        umur = a;
    }

    //method getter
    public int getUmur() {
```

```
        return umur;

    }

}
```

[DemoManusia]

```
/*
 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this license
 * Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Class.java to edit this template
 */
package andita.modul04.guided1.main;

import andita.modul04.guided1.manusia.*;
public class DemoManusia {
    public static void main(String[]args){
        Manusia arrMns[] = new Manusia[3]; // buat array of Object

        // constructor pertama
        Manusia objMns1 = new Manusia();
        objMns1.setNama("Markonah");
        objMns1.setUmur(76);
//constructor kedua

        Manusia objMns2 = new Manusia("Mat Conan");

//constructor ketiga
        Manusia objMns3 = new Manusia("Bajuri", 13);
        arrMns[0] = objMns1;
        arrMns[1] = objMns2;
        arrMns[2] = objMns3;
        for (int i = 0; i < 3; i++) {

            System.out.println("Nama : " + arrMns[i].getNama());
            System.out.println("Umur : " + arrMns[i].getUmur());
            System.out.println();
        }

    }
}
```

Output :

```
--------------------------< andita:modul04 >--------------------------
Building modul04 1.0-SNAPSHOT
  from pom.xml
--------------------------------[ jar ]---------------------------------

--- resources:3.4.0:resources (default-resources) @ modul04 ---
skip non existing resourceDirectory C:\Praktikum PBO\TemplatePBO\praktikum-pbo-ps1ds-05-reg-01\modul-04-enkapsulasi\project\ProjectModul04\modul04\src

--- compiler:3.15.0:compile (default-compile) @ modul04 ---
Nothing to compile - all classes are up to date.

--- exec:3.6.3:exec (default-cli) @ modul04 ---
Nama : Markonah
Umur : 76

Nama : Mat Conan
Umur : 0

Nama : Bajuri
Umur : 13

------------------------------------------------------------------------
BUILD SUCCESS
------------------------------------------------------------------------
Total time:  1.316 s
Finished at: 2026-10-12T19:39:31+07:00
```

2. [HargaPulsa]

```
package andita.modul04.guided2.hargapulsa;


public class HargaPulsa {

    public void info() {
        System.out.println("Harga Pulsa");
        System.out.println("Rp. 5000");
        System.out.println("Rp. 10000");
        System.out.println("Rp. 20000");
        System.out.println("Rp. 50000");
        System.out.println("Rp. 100000");
    }
}
```

[HargaToken]

```
package andita.modul04.guided2.hargatoken;

public class HargaToken {

    public void info() {
        System.out.println("Harga Token");
        System.out.println("Rp. 20000");
        System.out.println("Rp. 50000");
        System.out.println("Rp. 100000");
        System.out.println("Rp. 200000");
```

```
        System.out.println("Rp. 500000");
    }
}
```

[main]

```
package andita.modul04.guided2.main;

import andita.modul04.guided2.hargatoken.HargaToken;
import andita.modul04.guided2.hargapulsa.HargaPulsa;

public class main {

    public static void main(String[] args) {
        HargaToken objectToken = new HargaToken();
        objectToken.info();
        HargaPulsa objectPulsa = new HargaPulsa();
        objectPulsa.info();
    }
}
```

Output :

```
--- compiler:3.15.0:compile (default-compile) @ modul04 ---
Recompiling the module because of changed source code.
Compiling 6 source files with javac [debug release 26] to target\classes

--- exec:3.6.3:exec (default-cli) @ modul04 ---
Harga Token
Rp. 20000
Rp. 50000
Rp. 100000
Rp. 200000
Rp. 500000
Harga Pulsa
Rp. 5000
Rp. 10000
Rp. 20000
Rp. 50000
Rp. 100000
------------------------------------------------------------------------
BUILD SUCCESS
------------------------------------------------------------------------
Total time:  2.191 s
Finished at: 2026-10-12T19:47:20+07:00
------------------------------------------------------------------------
```

C. Unguided

1. Soal Unguided1

   Membuat program Java dengan struktur 3 package (model, laporan, dan main) untuk sistem pencatatan dataset: Dataset (package model): Menggunakan prinsip enkapsulasi, memiliki konstanta batas missing 5.0% counter total objek, 3 overloading constructor, serta method untuk menghitung persentase missing value dan status kebersihan data.

   LaporanDataset (package laporan): Memiliki method untuk mencetak format laporan informasi dataset secara rapi.

   Main (package main): Membuat 3 objek Dataset dengan constructor berbeda (tanpa parameter, 1 parameter, dan lengkap), menyimpannya dalam array, menampilkan laporannya menggunakan perulangan, serta mencetak total objek yang telah dibuat.

   Jawaban :

   [Dataset]

```
/*
 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this license
 * Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Class.java to edit this template
 */
package andita.modul04.unguided.model;


/**
 *
 * @author andit
 */
public class Dataset {
    // atribut (private = enkapsulasi)
    private String nama;
    private int jumlahBaris;
```

```
    private int jumlahKolom;
    private int jumlahMissing;

    // konstanta
    public static final double BATAS_MISSING = 5.0;

    // variabel milik class
    private static int totalDataset = 0;

    // constructor 1: tanpa parameter
    public Dataset() {
        totalDataset++;
    }

    // constructor 2: hanya nama
    public Dataset(String nama) {
        this.nama = nama;
        totalDataset++;
    }

    // constructor 3: lengkap
    public Dataset(String nama, int jumlahBaris, int jumlahKolom, int jumlahMissing) {
        this.nama = nama;
        this.jumlahBaris = jumlahBaris;
        this.jumlahKolom = jumlahKolom;
        this.jumlahMissing = jumlahMissing;
        totalDataset++;
    }
```

```
// method static untuk ambil total
public static int getTotalDataset() {
    return totalDataset;
}

// getter dan setter
public String getNama() {
    return nama;
}

public void setNama(String nama) {
    this.nama = nama;
}

public int getJumlahBaris() {
    return jumlahBaris;
}

public void setJumlahBaris(int jumlahBaris) {
    if (jumlahBaris >= 0) {
        this.jumlahBaris = jumlahBaris;
    }
}

public int getJumlahKolom() {
    return jumlahKolom;
}
```

```
public void setJumlahKolom(int jumlahKolom) {
    if (jumlahKolom >= 0) {
        this.jumlahKolom = jumlahKolom;
    }
}

public int getJumlahMissing() {
    return jumlahMissing;
}

public void setJumlahMissing(int jumlahMissing) {
    if (jumlahMissing >= 0) {
        this.jumlahMissing = jumlahMissing;
    }
}

// persentase missing
public double getPersentaseMissing() {
    int totalSel = jumlahBaris * jumlahKolom;
    if (totalSel == 0) {
        return 0;
    }
    return (double) jumlahMissing / totalSel * 100;
}

// perlu dibersihkan?
public boolean perluDibersihkan() {
```

```
        return getPersentaseMissing() > BATAS_MISSING;

    }

}
```

[Laporan]

```
package andita.modul04.unguided.laporan;

import andita.modul04.unguided.model.Dataset;
import java.util.Locale;

public class Laporan {

    public void cetak(Dataset d) {
        String status = d.perluDibersihkan() ? "Perlu dibersihkan" : "Bersih";

        System.out.println("=== Laporan Dataset ===");
        System.out.println("Nama         : " + d.getNama());
        System.out.println("Jumlah Baris : " + d.getJumlahBaris());
        System.out.println("Jumlah Kolom : " + d.getJumlahKolom());
        System.out.println(String.format(Locale.US, "Missing      : %d sel
(%.2f%%)",
                d.getJumlahMissing(), d.getPersentaseMissing()));
        System.out.println("Status       : " + status);
        System.out.println();
    }
}
```

[Main]

```
/*
 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to
change this license
 * Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Class.java to edit this
template
 */
package andita.modul04.unguided.main;

import andita.modul04.unguided.laporan.Laporan;
import andita.modul04.unguided.model.Dataset;

public class Main {

    public static void main(String[] args) {
```

```
        // objek 1: constructor kosong + setter
        Dataset d1 = new Dataset();
        d1.setNama("Titanic");
        d1.setJumlahBaris(891);
        d1.setJumlahKolom(12);
        d1.setJumlahMissing(866);

        // objek 2: constructor 1 parameter
        Dataset d2 = new Dataset("Wine Quality");

        // objek 3: constructor lengkap
        Dataset d3 = new Dataset("Iris", 150, 5, 0);

        // simpan dalam array
        Dataset[] daftar = {d1, d2, d3};

        // cetak dengan perulangan
        Laporan laporan = new Laporan();
        for (Dataset d : daftar) {
            laporan.cetak(d);
        }

        System.out.println("Total dataset dibuat : " + Dataset.getTotalDataset());
    }
}
```

Output :

```
] --- resources:3.4.0:resources (default-resources) @ modul04 ---
- skip non existing resourceDirectory C:\Praktikum PBO\TemplatePBO\praktikum-pbo-ps1d

] --- compiler:3.15.0:compile (default-compile) @ modul04 ---
  Recompiling the module because of changed source code.
- Compiling 9 source files with javac [debug release 26] to target\classes

] --- exec:3.6.3:exec (default-cli) @ modul04 ---
  === Laporan Dataset ===
  Nama         : Titanic
  Jumlah Baris : 891
  Jumlah Kolom : 12
  Missing      : 866 sel (8.10%)
  Status       : Perlu dibersihkan

  === Laporan Dataset ===
  Nama         : Wine Quality
  Jumlah Baris : 0
  Jumlah Kolom : 0
  Missing      : 0 sel (0.00%)
  Status       : Bersih

  === Laporan Dataset ===
  Nama         : Iris
  Jumlah Baris : 150
  Jumlah Kolom : 5
  Missing      : 0 sel (0.00%)
  Status       : Bersih

- Total dataset dibuat : 3
  ------------------------------------------------------------------------
  BUILD SUCCESS
  ------------------------------------------------------------------------
  Total time:  2.210 s
  Finished at: 2026-10-12T20:07:16+07:00
```

Penjelasan :

Hasil program menunjukkan bahwa dataset Titanic memiliki 891 baris dan 12 kolom sehingga total selnya adalah 10.692. Dengan 866 sel kosong, persentase missing-nya adalah 8,10%. Nilai ini melebihi batas 5%, sehingga statusnya "Perlu dibersihkan". Dataset Wine Quality dibuat hanya dengan nama sehingga jumlah baris, kolom, dan missing bernilai 0. Karena total selnya 0, persentase missing dikembalikan 0 dan statusnya "Bersih". Dataset Iris memiliki 150 baris dan 5 kolom (total 750 sel) tanpa sel kosong, sehingga persentase missing-nya 0% dan statusnya "Bersih". Baris terakhir, "Total dataset dibuat : 3", menunjukkan bahwa variabel `static` berhasil menghitung tiga objek yang dibuat melalui tiga constructor. Proses build juga berakhir dengan status BUILD SUCCESS, yang berarti program berhasil dikompilasi dan dijalankan tanpa error.

D. Kesimpulan

Berdasarkan praktikum Modul 4 tentang Enkapsulasi, dapat disimpulkan bahwa konsep enkapsulasi pada Java diterapkan dengan menjadikan atribut bersifat private dan menyediakan method public berupa getter dan setter sebagai antarmuka untuk mengaksesnya. Pada soal unguided, penerapan ini terlihat pada class Dataset, di mana setter dilengkapi validasi sehingga data yang tidak valid tidak dapat masuk ke dalam objek. Modifier final terbukti berguna untuk mendefinisikan konstanta BATAS_MISSING, sedangkan modifier static memungkinkan variabel totalDataset dipakai bersama oleh seluruh objek dan dapat diakses melalui method getTotalDataset() tanpa membuat objek terlebih dahulu. Constructor dengan konsep *overloading* juga memudahkan pembuatan objek dengan berbagai kondisi data, mulai dari objek kosong, objek dengan nama saja, hingga objek dengan data lengkap. Selain itu, penggunaan package membuat program lebih terstruktur karena class dikelompokkan sesuai tugasnya, yaitu model untuk data, laporan untuk tampilan, dan main untuk menjalankan program, dengan import digunakan untuk menghubungkan class antar package. Hasil program menunjukkan bahwa seluruh ketentuan soal terpenuhi: dataset Titanic dengan persentase missing 8,10% berstatus "Perlu dibersihkan", sedangkan Wine Quality dan Iris berstatus "Bersih", dan total dataset yang dibuat tercatat sebanyak 3. Dengan demikian, praktikum ini menunjukkan bahwa penerapan modifier, constructor, dan package secara tepat menghasilkan program yang lebih aman, rapi, dan mudah dikelola.